*** Why do you want to leave your current job?**

seeking better career growth and development, learning new skills and gaining new experience, seeking better job security or stability, wanting a better work-life balance, wanting to challenge oneself and grow in one's career, aligning with a company's values, pursuing personal growth, finding a better opportunity, needing to relocate for personal reasons

*** From your resume it seems you took a gap year. Would you like to tell us why that was?**

*** What are your salary expectations?**

*** What d u mean by Synchronized ?**

*** Is there any alternative for synchronization?**

*** What is alternative to synchronized in java**

*** How to show error message to client**

*** What methods used for sorting**

*** How to build a maven project and configuration**

*** Scenarios and Programs on multithreading**

*** What is printStackTrace**

The printStackTrace() method in Java is a tool used to handle exceptions and errors. It is a method of Java's throwable class which prints the throwable along with other details like the line number and class name where the exception occurred. printStackTrace() is very useful in diagnosing exceptions.

*** Write a program to print sequence of numbers using 4 threads**

*** What is Custom Exception? Write scenario of custom exception**

*** Write program for Custom Exception**

*** what is exception handling?write a scenario of exception handling?**

*** how do you handle exception**

*** Hierarchy of Java Exception classes**

*** what is io exception**

*** when do the get file not found exception**

FileNotFoundException is another exception class available in the java.io package. The exception occurs when we try to access that file which is not available in the system. It is a checked exception because it occurs at run time, not compile-time, and it is thrown by one of the following constructors:

1. RandomAccessFile
2. FileInputStream
3. FileOutputStream

*** Why FileNotFoundException occurs?**

There are mainly two reasons by which we get this error. The reasons for getting this exception are as follows:

When we try to access that file, that is not available in the system.

When we try to access that file which is inaccessible, for example, if a file is available for read-only operation and try to modify it, it can throw the error.

*** difference between collection and collections**

The **Collection** is an interface (extends iterable interface)whereas **Collections** is a **utility** class(extends Object class) in Java. The **Set, List,** and **Queue** are some of the subinterfaces of **Collection** interface, a **Map** interface is also part of the **Collections** Framework, but it doesn't inherit **Collection** interface. The important methods of **Collection** interface are **add(), remove(), size(), clear()** etc and **Collections** class contains only **static** methods like **sort(), min(), max(), fill(), copy(), reverse()** etc.

*** difference between final finally finalized**

The basic difference between final, finally and finalize is that the **final** is an access modifier, **finally** is the block in Exception Handling and **finalize** is the method of object class.

Sr. no.	Key	final	finally	finalize
1.	Definition	final is the keyword and access modifier which is used to apply restrictions on a class, method or variable.	finally is the block in Java Exception Handling to execute the important code whether the exception occurs or not.	finalize is the method in Java which is used to perform clean up processing just before object is garbage collected.
2.	Applicable to	Final keyword is used with the classes, methods and variables.	Finally block is always related to the try and catch block in exception handling.	finalize() method is used with the objects.
3.	Functionality	(1) Once declared, final variable becomes constant and cannot be modified. (2) final method cannot be overridden by sub class. (3) final class cannot be inherited.	(1) finally block runs the important code even if exception occurs or not. (2) finally block cleans up all the resources used in try block	finalize method performs the cleaning activities with respect to the object before its destruction.
4.	Execution	Final method is executed only when we call it.	Finally block is executed as soon as the try-catch block is executed. It's execution is not dependant on the exception.	finalize method is executed just before the object is destroyed.

* why do need finally?

* what is alternative of finally block in java

* What is the difference between get and load method?

No.	get()	load()
1)	Returns **null** if an object is not found.	Throws **ObjectNotFoundException** if an object is not found.
2)	get() method always **hit the database**.	load() method **doesn't hit** the database.
3)	It returns the real object, not the proxy.	It returns **proxy object.**
4)	It should be used if **you are not sure** about the existence of instance.	It should be used if **you are sure** that instance exists.

*** contract between equals and hashcode method**

*** Internal working of hashmap**

*** Difference between Comparable and Comparator**

Comparable and Comparator both are interfaces and can be used to sort collection elements.

Comparable	Comparator
1) Comparable provides a **single sorting sequence**. In other words, we can sort the collection on the basis of a single element such as id, name, and price.	The Comparator provides **multiple sorting sequences**. In other words, we can sort the collection on the basis of multiple elements such as id, name, and price etc.
2) Comparable **affects the original class**, i.e., the actual class is modified.	Comparator **doesn't affect the original class**, i.e., the actual class is not modified.
3) Comparable provides **compareTo() method** to sort elements.	Comparator provides **compare() method** to sort elements.
4) Comparable is present in **java.lang** package.	A Comparator is present in the **java.util** package.
5) We can sort the list elements of Comparable type by **Collections.sort(List)** method.	We can sort the list elements of Comparator type by **Collections.sort(List, Comparator)** method.

*** What is serialization?**

*** What is de Serialization?**

*** Explain java8 features.**

*** 16)write a step of hibernate configuration**

17)what is hibernat dialect?

18) what is pojo class? how to write pojo class why need pojo class

19)what is yml file

20) what is pom.xml file

22)explain hibernate architecture?

21)what is hibernate does?

22)what is hibernate annotation expalin each

23)what is flow of hibernate to create and application?

10. In Hibernate, use of cascade

- why string is immuatable?

- What is immutable class?

- can we create a immuatable class? if yes the how?

- how to serialize an object without implementing serializable interface.

*** Map is part of collection framework or not?**

Map interface is also part of Java collection framework but it does not inherit the Collection interface.

*** In Hibernate mapping Which one is best suitable for many type - set or list?**

the choice between using a Set or a List for mapping a collection of entities in Hibernate depends on the specific requirements of the use case. **If the order of the elements in the collection is important, then a List is the best choice. If the collection should not contain duplicates, then a Set is the best choice.** However, the type also influences how Hibernate manages the association in the database, so the choice should also take into account the efficiency of the mapping.

*** what is one to many and many to many mapping ?**

One-to-many mapping refers to a relationship between two entities where one entity can have multiple instances of another entity, but the second entity can only have one instance of the first entity. For example, an employee can work in one division, but a division can have multiple employees.

Many-to-many mapping refers to a relationship between two entities where one entity can have multiple instances of another entity, and the second entity can also have multiple instances of the first entity. An example of this is the relationship between students and classes, where a student can take multiple classes, and a class can have multiple students.

*** What is jhipster?**

JHipster is a free and open-source development platform that generates modern web applications and microservices using Angular or React and the Spring Framework.

*** many people say that set is most used than list what u say?**

The choice between using a Set or a List depends on the specific use case. If you need to maintain the order of elements and allow duplicates, then a List is the better choice. If you need to ensure that there are no duplicates and order is not important, then a Set is the better choice

*** is a sort method takes set or not?**

Collections.sort() method can take a List as an argument, but it cannot take a Set directly.

it is possible to convert a Set to a List and then sort the List using Collections.sort(). or

there are some Set implementations that maintain the order of elements, such as TreeSet.

*** can we override static method or not?**

No, we cannot override static methods in Java. This is because static methods are associated with the class rather than the object, and they resolved using Static Binding during compile time. On the other hand, method overriding is based on dynamic binding at runtime, where the correct overridden method is resolved based on the object at runtime.

However, we can declare static methods with the same signature in a subclass, but it is not considered overriding as there won't be any runtime polymorphism. If a derived class defines a static method with the same signature as a static method in the base class, the method in the derived class is hidden by the method in the base class.

In summary, we cannot override static methods in Java, but we can declare static methods with the same signature in a subclass, which is not considered overriding.

*** Can we Overload static method in Java**

Yes, we can overload the static method in Java. In terms of method overloading, the static method is just like normal methods. To overload the static method, you need to provide another static method with the same

name but a different method signature. The static overloaded method is resolved using Static Binding during compile time. The overloading method in Java is completely different than the overriding method.

*** Which collection classes are thread-safe in Java?**

The collection classes that are **thread-safe** in Java
are **Stack**, **Vector**, **Properties**, **Hashtable**, ConcurrentHashMap etc.

*** is hashmap is thread safe?**

No, HashMap is not thread-safe.

it is possible to synchronize a HashMap using
the Collections.synchronizedMap() method.

(It is important to note that Hashtable is synchronized and can be shared with many threads.)

*** concurrent hashmap is thread safe or not ? & what he do for thread safety?**

OR

*** can a multiple thread access same object in concurrent hashmap?**

ConcurrentHashMap is thread-safe, which means that multiple threads can access it simultaneously without any synchronization issues.

The ConcurrentHashMap class achieves thread-safety by dividing the map into segments, the lock is required not for the entire object but for one segment, i.e one thread requires a lock of one segment.

***what is Linked hashmap?**

Java LinkedHashMap class is Hashtable and Linked list implementation of the Map interface, with predictable iteration order. It inherits HashMap class and implements the Map interface.

***Which implementation is better to use: HashMap or LinkedHashMap?**

Both HashMap and LinkedHashMap classes provide comparable performance but HashMap is a natural choice if the ordering of elements is not an issue.

Adding, removing, and finding entries in a LinkedHashMap is slightly slower than in HashMap because it needs to maintain the order of doubly linked list in addition to the hashed data structure.

***what u think which gives best performance linked list and arraylist?**

ArrayList	**LinkedList**

It is best choice if our frequent operation is retrieval.	**It is the best choice if our frequent operation is insertion and deletion.**
ArrayList provides random access.	**LinkedList does not provide random access.**
Underlying data structure for arraylist is resizable or growable array.	**Underlying data structure is double linked list.**
ArrayList takes less memory as it stores only object	**LinkedList takes more memory, as it stores the object as well as the address of that object.**

***what are pros and cons of wrapper classes.**

Pros:

Wrapper classes provide a way to use primitive data types as objects, which is required in certain situations, such as when working with collections that can only store objects

Wrapper classes provide a way to convert primitive data types into objects and vice versa

Cons:

they are less efficient than primitive data types and can lead to more complex code and errors if not used properly.

***Differnce between global variable vs local variable**

Global Variable	**Local Variable**
It is defined outside of a method but inside the class. It is called as instance variable.	**The variables which are declared inside method body, constructor or block**
It stored in "Heap Memory area", each instance(object) of a class has its own copy of instance variable.	**Local variables are stored in stack area.**
It is initialized automatically by JVM and default value is zero.	**It does not initialize automatically. If you are trying to use local variables without initialization then we will get compile time error.**

Global variables scope is anywhere in the class.	**Scope of local variables is within the method, constructor and block only.**
Static keyword can be applied to global variable and it will become static variable.	**Static keyword is not applied to local variable**

*** what is default priority, min priority, max priorty of thread ?**

thread's priority is an integer value in the range 1 to 10 .Default priority of a thread is 5 (NORM_PRIORITY). The value of MIN_PRIORITY is 1 and the value of MAX_PRIORITY is 10.

*** can we call run method by its name in multithreading?**

Yes, we can call the run() method by its name in multithreading, but it will not create a new thread.

The run() method is the entry point for a new thread.

To create a new thread in Java, we need to create an instance of the Thread class and pass an instance of a class that implements the Runnable interface to its constructor. The Runnable interface has only one method, run(), which is called when the thread starts.

*** what are ways to create thread & which is the best way.**

there are two ways to create a thread:

by extending the Thread class and by implementing the Runnable interface.

Extending thread class- if the class is not extending another class then we should go for thread class.

Implementing runnable interface- if our class is already extending another class then we could not use extend keyword due to multiple inheritance. So best way to go for runnable interface.

*** When we implement runnable r u sure we create thread class object & then why we create thread class object**

when we implement the Runnable interface, we do not create a Thread class object directly. Instead, we create a Thread object and pass the Runnable object to its constructor.

```java
public class MyRunnable implements Runnable {
  public void run() {
    System.out.println("MyRunnable running");
  }
}

public class Main {
  public static void main(String[] args) {
    MyRunnable myRunnable = new MyRunnable();
    Thread thread = new Thread(myRunnable);
    thread.start();
  }
}
```

*** have u work with spring security**

Spring Security is a framework which provides various security features like: authentication, authorization to create secure Java Enterprise Applications.

Authentication is the process of identifying a user to provide access to a system.

In this, the user or client and server are verified.

Authorization is the process of giving permission to access the resources.

In this, it is verified that if the user is allowed through the defined policies and rules.

*** how many types of cache in java**

Hibernate provides two types of caching: First Level Cache and Second Level Cache and Query Cache.

No.	First Level Cache	Second Level Cache
1)	First Level Cache is **associated with Session**.	Second Level Cache is associated with **SessionFactory**.
2)	It is **enabled** by default.	It is **not enabled** by default.

*** what is query level cache and when it excute**

Query cache will cache the results of the query against the object.

If we have queries which runs multiple times with the same parameters then Query caching is best to use to avoid multiple DB calls.

Query cache is disabled by default and we can enable it using configuration.

We just need to set the hibernate.cache.use_query_cache property to true to enable Query cache.

*** What is JPA?**

A JPA (Java Persistence API) is a specification of Java which is used to access, manage, and persist data between Java object and relational database. It is considered as a standard approach for (ORM)Object Relational Mapping.

JPA can be seen as a bridge between object-oriented domain models and relational database systems. Being a specification, JPA doesn't perform any operation by itself. Thus, it requires implementation. So, ORM tools like Hibernate, TopLink, and iBatis implements JPA specifications for data persistence.

*** What Is a JPA Repository in Spring Boot?**

JPA Repository is mainly used for managing the data in a Spring Boot Application.

*** What is Hibernate?**

Hibernate is an open-source and lightweight ORM tool that is used to store, manipulate, and retrieve data from the database.

Hibernate framework is a mediator through which java application can communicate with database.

Hibernate is an implementation of JPA. So, it follows the common standards provided by the JPA.

*** Need of JPA**

As we have seen so far, JPA is a specification. **It provides common prototype and functionality to ORM tools.** By implementing the same specification, all ORM tools (like Hibernate, TopLink, iBatis) follows the common standards. In the future, if we want to switch our application from one ORM tool to another, we can do it easily.

*** JPA vs. Hibernate**

JPA	Hibernate
Java Persistence API (JPA) defines the management of relational data in the Java applications.	Hibernate is an Object-Relational Mapping (ORM) tool which is used to save the state of Java object into the database.
It is just a specification. Various ORM tools implement it for data persistence.	It is one of the most frequently used JPA implementation.
It is defined in **javax.persistence** package.	It is defined in **org.hibernate** package.
The **EntityManagerFactory** interface is used to interact with the entity manager factory for the persistence unit. Thus, it provides an entity manager.	It uses **SessionFactory** interface to create Session instances.
It uses **EntityManager** interface to create, read, and delete operations for instances of mapped entity classes. This interface interacts with the persistence context.	It uses **Session** interface to create, read, and delete operations for instances of mapped entity classes. It behaves as a runtime interface between a Java application and Hibernate.
It uses **Java Persistence Query Language** (JPQL) as an object-oriented query language to perform database operations.	It uses **Hibernate Query Language** (HQL) as an object-oriented query language to perform database operations.

*** is project developed from scratch(start)**

*** which technology used in project**

*** tell Project Desciption**

***Solid principles-explain each of them.**

SOLID principles are an object-oriented approach that are applied to software structure design

The word SOLID acronym for:

- Single Responsibility Principle (SRP)
- Open-Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)

*** Can we override or replace the Embedded tomcat server in Spring Boot?**

Yes, we can replace the Embedded Tomcat server with any server by using the Starter dependency in the **pom.xml file**. Like you can use **spring-boot-starter-jetty** as a dependency for using a jetty server in your project.

***SpringFramework**

It is a lightweight, loosely coupled and integrated framework for developing enterprise applications in java.

***SpringBoot**

It is a Spring module that provides the **RAD (***Rapid Application Development***)** feature to the Spring Framework.

It is used to create a stand-alone Spring-based application that you can just run because it needs minimal Spring configuration.

***Difference between Spring Framework and Spring Boot.**

S.No.	Spring	Spring Boot
1.	Spring is an open-source lightweight framework widely used to develop enterprise applications.	Spring Boot is built on top of the conventional spring framework, widely used to develop REST APIs.
2.	The most important feature of the Spring Framework is dependency injection.	The most important feature of the Spring Boot is Autoconfiguration.
3.	It helps to create a loosely coupled application.	It helps to create a stand-alone application.
4.	To run the Spring application, we need to set the server explicitly.	Spring Boot provides embedded servers such as Tomcat and Jetty etc.
5.	To run the Spring application, a deployment descriptor is required.	There is no requirement for a deployment descriptor.
6.	To create a Spring application, the developers write lots of code.	It reduces the lines of code.
7.	It doesn't provide support for the in-memory database.	It provides support for the in-memory database such as H2.
8.	Developers need to write boilerplate code for smaller tasks.	In Spring Boot, there is reduction in boilerplate code.
9.	Developers have to define dependencies manually in the pom.xml file.	pom.xml file internally handles the required dependencies.

*****

***What New Features Were Added in Java 8?**

1) Lambda Expression

2) Functional Interfaces

3) Default methods

4) Predicates

5) Optional Class

6) Double colon operator (::)

7) Stream API

8) Date and Time API

***What is a Predicate? State the difference between a Predicate and a Function?**

Predicate is a pre-defined Functional Interface. It is under java.util.function package. It accepts only a single argument

Predicate	Function
To implement conditional checks We should go for predicate	To perform certain operation And to return some result we Should go for function.
Predicate can take one type Parameter which represents Input argument type. Predicate<T>	Function can take 2 type Parameters. First one represent Input argument type and Second one represent return Type. Function<T,R>
Predicate interface defines only one method called test()	Function interface defines only one Method called apply().
public boolean test(T t)	public R apply(T t)
Predicate can return only boolean value.	Function can return any type of value

<u>Note:</u> Predicate is a boolean valued function and(), or(), negate() are default methods present inside Predicate interface.

***What are the main characteristics of the Lambda Function?**

Lambda Expression is just an anonymous (nameless) function. That means the function which doesn't have the name, return type and access modifiers.

-A lambda expression can have zero or more number of parameters (arguments)

-Usually we can specify type of parameter. If the compiler expects the type based on the context then we can remove type. i.e., programmer is not required.

-If multiple parameters present then these parameters should be separated with comma (,).

-If zero number of parameters available then we have to use empty parameter [like ()].

-If only one parameter is available and if the compiler can expect the type then we can remove the type and parenthesis also.

-Similar to method body lambda expression body also can contain multiple statements. If more than one statements present then we have to enclose inside within curly braces. If one statement present then curly braces are optional.

-Once we write lambda expression we can call that expression just like a method, for this functional interfaces are required.

***What are Functional Interfaces?**

If an interface contain only one abstract method, such type of interfaces are called functional interfaces and the method is called functional method or single abstract method (SAM).

Ex:

1) Runnable → It contains only run() method

2) Comparable → It contains only compareTo() method

3) ActionListener → It contains only actionPerformed()

4) Callable → It contains only call() method

Inside functional interface in addition to single Abstract method (SAM) we write any number of default and static methods.

In Java 8, Sun Micro System introduced @Functional Interface annotation to specify that the interface is Functional Interface.

***How can you create a Functional Interface?**

Although Java can identify a Functional Interface, you can define one with the annotation @FunctionalInterface

* What is a Stream API? Why do we require the Stream API?

Stream API is a new feature added in Java 8. It is a special class that is used for processing objects from a source such as Collection.

We require the Stream API because,

- It supports aggregate operations which makes the processing simple.
- It supports Functional-Style programming.
- It does faster processing. Hence, it is apt for better performance.
- It allows parallel operations.

*What is the difference between the Collection API and Stream API?

Stream API	Collection API
It was introduced in Java 8 Standard Edition version.	It was introduced in Java version 1.2
There is no use of the Iterator and Spliterators.	With the help of forEach, we can use the Iterator and Spliterators to iterate the elements and perform an action on each item or the element.
An infinite number of features can be stored.	A countable number of elements can be stored.
Consumption and Iteration of elements from the Stream object can be done only once.	Consumption and Iteration of elements from the Collection object can be done multiple times.
It is used to compute data.	It is used to store data.

*If we have HashTable for thread safe operation, then what is the need to go for "Concurrent HashMap"

Sr. No.	Key	HashTable	ConcurrentHashMap
1	Basic	HashTable is a thread-safe legacy class introduced in the Jdk1.1	ConcurrentHashmap is a class that was introduced in jdk1.5
2	Locking	It applies lock on the entire collection	ConcurrentHashMap apply locks only at bucket level called fragment while adding or updating the map
3	Performance	It is slower than ConcurrentHashMap	It is better than HashTable
4.	Null	It doesn't allow null key and value	It allows null key and value

* Difference between fail fast and fail safe with example

FAIL-FAST ITERATOR	FAIL-SAFE ITERATOR
Fail-Fast Iterators **does not allow** us to make any **modification** to the collection while iterating	**Fail-Safe** Iterators **allows** us to **modify** to the collection while iterating
Throws **ConcurrentModificationException** when the collection is modified	**Does not** throw **ConcurrentModificationException** when the collection is modified
Uses the **original collection** for the iteration	Uses the **snapshot of the collection** for the iteration
We are allowed to **modify the collection** using the **iterator instance**	We are **not allowed** to modify the collection using the **iterator instance**
It **does not** need **additional memory** as the iteration happens on the original collection	Requires **additional memory**, as it takes a **snapshot** of the **original collection**
Example: Iterators of **ArrayList, HashMap, LinkedList, Vector**	**Example:** Iterators of **CopyOnWriteArrayList, ConcurrentHashMap, ConcurrentMap**

***How to handle "Concurrent Modification Exception"- write a code.**

we have an ArrayList of integers and we are trying to remove an element from the list while iterating over it. This will result in a ConcurrentModificationException being thrown.

```
import java.util.ArrayList;
import java.util.Iterator;

public class ConcurrentModificationExample {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);
        list.add(4);
        list.add(5);

        Iterator<Integer> iterator = list.iterator();
        while (iterator.hasNext()) {
            Integer element = iterator.next();
            if (element == 3) {
                list.remove(element);
            }
        }
    }
}
```

To avoid this exception, we can use an iterator to iterate over the list and remove the element using the remove() method of the iterator instead of the remove() method of the list.

```
import java.util.ArrayList;
import java.util.Iterator;

public class ConcurrentModificationExample {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(3);
        list.add(4);
        list.add(5);

        Iterator<Integer> iterator = list.iterator();
        while (iterator.hasNext()) {
            Integer element = iterator.next();
            if (element == 3) {
                iterator.remove();
            }
        }
    }
}
```

* What are the types of IOC container in spring?

There are two types of IOC containers in spring framework.

1. BeanFactory
2. ApplicationContext

*What is the difference between constructor injection and setter injection?

No.	Constructor Injection	Setter Injection
1)	No Partial Injection	Partial Injection
2)	Desn't override the setter property	Overrides the constructor property if both are defined.
3)	Creates new instance if any modification occurs	Doesn't create new instance if you change the property value
4)	Better for too many properties	Better for few properties.

More details...

* What is the difference between BeanFactory and ApplicationContext?

ApplicationContext extends the BeanFactory interface.BeanFactory is the **basic container** whereas ApplicationContext is the **advanced container**. ApplicationContext provides more facilities than BeanFactory such as integration with spring AOP, message resource handling

***What is Spring MVC**

A Spring MVC is a Java framework which is used to build web applications.

It follows the Model-View-Controller design pattern.

It implements all the basic features of a core spring framework like Inversion of Control, Dependency Injection

It is build on the top of Servlet API

* What does the ViewResolver class?

The View Resolver class resolves the view component to be invoked for the request. It defines prefix and suffix properties to resolve the view component.

* Which ViewResolver class is widely used?

The org.springframework.web.servlet.view.**InternalResourceViewResolver** clas s is widely used.

* What is the front controller class of Spring MVC?

Front Controller - In Spring Web MVC, the **DispatcherServlet** class works as the front controller. It is responsible to manage the flow of the Spring MVC application.

*

*

*

*

*